CS224 - Fall 2025- Lab #5 (Version 1, Dec. 7, 18:46)

Cache Memory Principles & Practicing Memory Mapped I/O[1]

This lab has two independent parts: Cache Principles (Part A) and Memory Mapped I/O (Part B).

You can do this lab in GROUPS of 2 or INDIVIDUALLY. There are changes in submission requirements for preliminary report and lab work. You must read the entire document carefully.

If done in groups each student must submit a copy (my partner submitted I forgot to submit is not an excuse: take care your own submission). Student can have partners only from their own section.

Dates: (TAs)
Section 1: Wed, 17 Dec, 13:30-17:20 (Kadri, Mert, Omid, Yağız)
Section 2: Thu, 18 Dec, 13:30-17:20 (Berkan, Kadri)
Section 3: Thu, 18 Dec, 8:30-12:20 (Mahyar, Mert, Tuna, Yağız)
Section 4: Fri, 19 Dec, 13:30-17:20 (Mahyar, Omid, Tuna)

TA Full Name (email address: @bilkent.edu.tr) No of Labs
Berkan Şahin: (berkan.sahin) 1
Kadri Gofralılar: (kadri.gofralilar) 2
Mahyar Fardinfar (mahyar.fardinfar) 2
Mert Barkın Er (barkin.er) 2
Omid Feizi: (omid.feizi) 2
Sepehr Bakhshi: (sepehr.bakhshi) *
Tuna Okçu (tuna.okcu) 2
Yağız Özkarahan (yagiz.ozkarahan) 2
* Coordinating TA

Summary of Work to be Performed

This lab has two parts.
Part A: Cache Memory Principles
Part B: Memory Mapped I/O: Self Study

Part A

Purpose: Learning Cache Principles

In this part you will study the effects of various cache design parameters. In the preliminary component,

[1] The Moodle Lab folder contains a problem set on cache memory. It is for self-study you do not need to submit anything for those problems.

you will write a program. In the lab component you will execute the program (with possible extensions etc. if suggested by your TA) and preparing a report.

Preliminary Work: 20 points
Writing a matrix addition program with a simple user interface.
Lab Work: 30 points
Experimental observations using a program that finds the summation of the elements of a matrix with different cache conditions.

In your report make sure that you use properly numbered tables and figures. All tables must have subtitle and table number furthermore columns must have column names, etc. In your report make sure that you have a nice presentation. Make sure that you number the pages. Try to do everything before coming to the lab but make sure that you demonstrate your work to your TA.

Part B
Purpose: Practicing Memory Mapped I/O (Self Learning)
In this part will use the C programming language to develop simple applications for the PIC32 microcontroller. The mikroC IDE (integrated development environment) will be used as the software environment, and the Beti PIC32 Training Set card will be used as the hardware environment in this lab. This lab involves self-learning.

Preliminary Work: 15 points
Writing a C program that performs I/O using mikro C IDE.
Lab Work: 35 points
Experiment with the C program.

Summary of What to Upload for Preliminary and Lab Work of Part A & B

Part	Preliminary Work	Lab Work
A. Cache Memory Principles Prelim.: 20 pts Lab: 30 pts	**1.** MIPS Assembly Language Program	**1.** MIPS Assembly Language Program (with change or no change) **2.** Report (pdf)
B. Memory Mapped I/O Prelim.: 15 pts Lab: 35 pts	**1.** C Program	**1.** C Program (with change or no change)

Important Notes for All Labs About Attendance, Performing and Presenting the Work
You are obliged to read this document word by word and are responsible for the mistakes you make by not following the rules.

1. Not attending to the lab means 0 out of 100 for that lab. If you attend the lab but do not submit the preliminary part you will lose only the points for the preliminary part.

2. Try to complete the lab part at home before coming to the lab. Make sure that you show your work to your TAs and answer their questions to show that you know what you are doing before uploading your lab work and follow the instructions of your TAs.
3. In all labs if you are not told you may assume that inputs are correct.
4. In all labs when needed you have to provide a simple user interface for inputs and outputs.
5. Presentation of your work
   You have to provide a neat presentation prepared in txt form. Your programs must be easy to understand and well structured.
   Provide following six lines at the top of your submission for preliminary and lab work (make sure that you include the course no. CS224, important for ABET documentation).

   CS224
   Lab No.
   Section No.
   Your Full Name
   Bilkent ID
   Date

   Please also make sure that your work is identifiable: In terms of which program corresponds to which part of the lab.
6. **If we suspect that there is cheating, we will send the work with the names of the students to the university disciplinary committee. You can experiment with ChatGPT for learning; however, you cannot use/modify the code generated by it. Such an act is classified as plagiarism. Note that MOSS is capable of detecting ChatGPT code. Furthermore, remember that, the code you use from ChatGPT can also be used by another student in the course. Make sure that the code you submit is really yours and has been internalized.**
7. Note that the programs you upload for lab part can be identical with the preliminary part but you have to upload both. This is a requirement. If not done you will lose the lab part.

Due Date of Preliminary Work: Same for All Sections

No late submission will be accepted.

a. Please upload your programs of preliminary work to Moodle by **13:30 on Wednesday, 17 December, 2025.**

b. Please note that the submission closes sharp at 13:30 and no late submissions will be accepted. You can make resubmissions so do not wait for the last moment. Submit your work earlier and change your submitted work if necessary. Note that only the last submission will be graded.

c. For you preliminary work to be graded you have to submit the Lab Work part.

d. Please familiarize yourself with the Moodle course interface, find the submission entry early, and avoid sending an email like "I cannot see the submission interface." (As of now it is not yet opened.)

e. Do not send your work by email attachment they will not be processed. They have to be in the Moodle system to be processed.

f. Preliminary Work Submission

Part A: Cache Principles

Submit your MIPS assembly language program for matrix summation.

StudentID_FirstName_LastName_SecNo_PRELIM_PartA_LabNo.txt [A NOTEPAD FILE as its extension suggests, which contains only the program part Only a NOTEPAD FILE (txt file) is accepted. Any other form of submission receives 0 (zero)

Part B: I/O Programming

Submit your C program.

StudentID_FirstName_LastName_SecNo_PRELIM_PartB_LabNo.txt [A NOTEPAD FILE as its extension suggests, which contains only the program part Only a NOTEPAD FILE (txt file) is accepted. Any other form of submission receives 0 (zero)

Do not send your work by email attachment, they will not be processed. They have to be in the Moodle system to be processed.

1. Preliminary and Lab Work for Cache Principles

Preliminary Work: Writing a MIPS Program for Matrix Summation

(20 points) Write a program to find the summation of the elements of a square matrix. Provide a user interface for user interaction to demonstrate that your program is working properly. Assume that in the main memory matrix elements are placed column by column. Create an array for the matrix elements and initialize them column by column with consecutive values. For example, a 3 by 3 (N= 3) matrix would have the following values.

1	4	7
2	5	8
3	6	9

The column-by-column placement means that you will have the values of the above 3 x 3 matrix are stored as follows in the memory.

Matrix Index (Row No., Col. No.)	(1, 1)	(2, 1)	(3, 1)	(1, 2)	(2, 2)	(3, 2)	(1, 3)	(2, 3)	(3, 3)
Displacement With respect the beginning of the array containing the matrix	0	4	8	12	16	20	24	28	32
Value stored	1	2	3	4	5	6	7	8	9

In this configuration, accessing the matrix element (i, j) simply involves computation of its displacement from the beginning of the array that stores the matrix elements. For example, the displacement of the matrix element with the index (i, j) with respect to the beginning of the array is (j - 1) x N x 4 + (i - 1) x 4, for a matrix of size N x N.

Your user interface must provide at least the following functionalities,

1. Ask the user the matrix size in terms of its dimensions (N),
2. Allocate an array with proper size using syscall code 9,
3. Obtain summation of matrix elements row-major (row by row) summation,
4. Obtain summation of matrix elements column-major (column by column) summation,
5. Display desired elements of the matrix by specifying its row and column member.

Lab Work: Cache Experiments with Matrix Addition Program

(30 points) Run your program with two reasonably large different matrix sizes that would provide meaningful observations. Modify your original program if needed. For large matrix initialization you may use a loop rather than user interface.

Report for Matrix Size 1: 15 Points

Report for Matrix Size 2: 15 Points

As described above make sure that you have a easy to follow presentation with numbered tables having proper heading etc.

Make sure that you find the summation of matrix elements by performing row-major and column-major addition. Note that the column-major addition is a simple array addition from the beginning to the end; however, the row-major addition is somewhat tricky.

a) **Direct Mapped Caches**: For the matrix sizes you have chosen, conduct tests with various cache sizes and block sizes, to determine the hit rate, miss rate and number of misses. Use at least 5 different cache sizes and 5 different block sizes (make sure your values are reasonable) in order to obtain curves like those of Figure 8.18 (see below) in the textbook. Make a 5 x 5 table with your values, with miss rate and # of misses as the data at each row-column location. Make a graph of miss rate versus block size, parameterized by cache size, like Figure 8.18.

b) **Fully Associative Caches**: Pick 3 of your parameter points obtained in part for row-major addition a), one with good hit rate, one with medium hit rate, and one with poor hit rate. For these 3 results, there were 3 configuration pairs of cache size and block size that resulted in the data. Take the same 3 configuration pairs, but this time run the simulation with a fully associate cache, using LRU replacement policy. Compare the results obtained: the Direct Mapped good result versus the Fully Associative good result, the Direct Mapped medium result versus the Fully Associative medium result, and the Direct Mapped poor result versus the Fully Associative poor result. How much difference did the change to fully associative architecture make? Now change the replacement policy to Random replacement, and run the 3 tests again (using the same 3 configuration pairs). Does replacement policy make a significant difference? Record these 9 values in a new table, with 3 lines: for Direct Mapped, for Fully Associative-LRU and for Fully Associative-Random.

c) **N-way Set Associative Caches**: to save on hardware costs, fully set-associative caches are rarely used. Instead, most of the benefit can be obtained with an N-way set associative cache. Pick the medium hit rate configuration that you found in a) and used again in b), and change the architecture to N-way set associative. For several different set sizes (at least 4) and LRU replacement policy, run

the program and record the hit rate, miss rate and number of misses. What set size gives the best result? How much improvement is gained as N (the number of blocks in a set) increases each step? Now repeat the tests, but for the good hit rate configuration from a) and b). Record these data and answer the same question again. Finally, repeat for the poor hit rate configuration.

After your report, try to deepen or expand the interpretation of your results by using a Generative AI tool such as ChatGPT—for example, by asking it to suggest insights or alternative explanations.

What kind of insight you get from Gen AI tool?
Is it useful or misleading? Any help at all? Explain.
NOTE THAT: Make sure that Gen AI part of your report has a proper section header to distinguish it from your explanation.

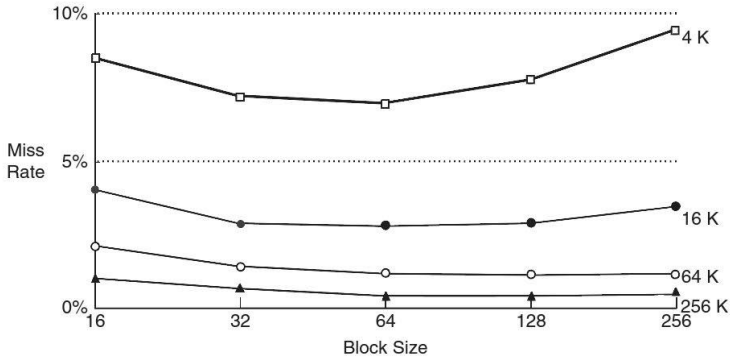

Figure 8.18 **Miss rate versus block size and cache size on SPEC92 benchmark**
Adapted with permission from Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed., Morgan Kaufmann, 2012.

Oral Interview with TA and Submission of your Data

Get ready for the interview with your TA, by gathering and analyzing your data, having it ready to submit in a clean understandable format. Be sure that you understand what you have done, and can interpret your data to the TA. Then call him over, and answer the questions he asks you.

2. Preliminary and Lab Work for Memory Mapped I/O

Preliminary Work: C Program

(15 points) C Program with DC Motor and Seven-Segment Modules

C code for DC Motor (read the lab part below), with lots of comments, an explanatory header, well-chosen identifiers and good use of spacing and layout to make your program self-documenting.

C code for Seven-Segment Module (read the lab part below), with lots of comments, an explanatory header, well-chosen identifiers and good use of spacing and layout to make your program self-documenting.

You can read Chapter 8.6 Embeded I/O Systems at textbook (particularly 8.6.2 General-Purpose Digital I/O section) and look into ***pic32IOPorts.pdf*** for specifying SFR. You will only use TRISx, PORTx and alternatively LATx registers during this lab.

Lab Work: I/O Experiments

(35 points) Run and if necessary, debug your C program.

Compiling, downloading and testing: 5 points

In this part you will learn to use mikroC IDE tool for the C language, targeting the PIC32 microcontroller. You will compile some simple code, use Beti PIC32 Training Set Card as the hardware environment, and download the code to program PIC32 microcontroller. Finally, you will develop a simple application program, and test your code on the board. If this is the first time you are doing this, you may ask your TA for help.

You will find the mikroC IDE development tool already installed on the lab computers. Use mikroC to implement the C program given in the ***ExampleProject.rar*** folder. You should compile and build the code and get its final output. For each build, the final output of mikroC will be a file with '.hex' extension. When you have made the hex file from the C program, you are ready to download it to PIC32. After successfully generating this hex file, use 'mikroBootloader' program which is also provided in ***ProgrammerTools.zip*** to program PIC32 microcontroller with the hex file. Check the manual provided in the zip file for help in mikroBootloader. You can find the 'mikroelektronika USB HID BootLoader v2.1.1.0' already on the lab computers.

About the Beti PIC32 Egitim Seti: Watch the Video

You only need to connect USB cable to small PIC32 daughter board for both power supply and programming. Please check the corresponding schematic file of Beti board module in ***Schematics.zip*** if you need more information. The number of the microcontroller we use is PIC32MX795F512L. You can refer to its datasheet (provided under ***beti.zip***) if you need more information. Additionally, you should watch the video ***beti_intro.mp4*** to learn about wiring and jumper connections.

Note that you borrow a Lab-board containing the development board, connectors, etc. in the beginning. The lab supervisor takes your signature. You are responsible for the lab board and you have to return it to the lab supervisor when you are done.

mikroC IDE Use Problems: Watch the Video

As you use mikroC IDE if you face a problem which requires admin privileges to solve, please use the method defined in the following video to bypass that problem:

https://www.youtube.com/watch?v=dP6wAjL_b1Q

DC Motor : 15 points

Use two pushbutton switches to specify the direction of the DC motor. You must choose button 0 and button 1 on the Beti training board. When button 0 is equal to 1, the motor should start turning clockwise one second after the button push, and when button 1 is equal to 1 it should be turning counterclockwise after one second. Motor must stop turning after 1 seconds. At any moment, when both buttons are pressed, the motor should not turn.

Hint: Check push button and DC Motor schematics before starting. In these schematics, J ports are the ones that you connect with a wire. Moreover, understanding the code given in example project will help you a lot in this part, as you will use SFR's in a similar manner.

Seven-Segment Module: 15 points

Using 4-digit Seven-Segment module, show the Fibonacci numbers. Before showing the **i**th Fibonacci number **x**, wait **x*100** ms. If you implement your code correctly, the display will appear like the box below. You can use the code in ***seven_segment.c*** as a starting point.

Hint: Check seven-segment module schematics before starting. Note that you can display only one digit at a certain time. However, if you switch digits fast enough, you can give the illusion that all digits are displayed simultaneously. An example of this is included in ***seven_segment.c***

```
  1
(100 milliseconds later)
  1
(200 milliseconds later)
  2
(300 milliseconds later)
  3
(500 milliseconds later)
  5
(800 milliseconds later)
  8
(1300 milliseconds later)
 13
(2100 milliseconds later)
 21
(...)
```

3. Submitting Your Lab Work

1. **Part A**: Cache Principles
   Submit your matrix summation MIPS assembly language program. If it is the same as your preliminary work you must still submit it.
   Submit your report.

   MIPS Program: StudentID_FirstName_LastName_SecNo_LAB_PartA_LabNo.txt [A NOTEPAD FILE as its extension suggests, which contains only the program part Only a NOTEPAD FILE (txt file) is accepted. Any other form of submission receives 0 (zero)

   Report: Put your experiment results (including the tables and graphs) in a single PDF file. Use filename **StudentID_FirstName_LastName_SecNo_LAB_ PartA_lab_report.pdf** [pdf FILE as its extension suggests, which contains all the work done for the Lab Experiment Report Part]

   Put your experiment results in a single PDF file. Use filename which contains all the work done for the Cache Experiments.

   Part B: I/O Programming
   Submit your C program program. If it is the same as your preliminary work you must still submit it.

   C Program: StudentID_FirstName_LastName_SecNo_LAB_PartB_LabNo.txt [A NOTEPAD FILE as its extension suggests, which contains only the program part Only a NOTEPAD FILE (txt file) is accepted. Any other form of submission receives 0 (zero)

2. Your programs will be compared against all the other programs in the class, by the MOSS program, to determine how similar it is (as an indication of plagiarism). So be sure that the code you submit is code that you actually wrote yourself! The same type of comparison is also planned for the reports.
3. *Even if you didn't finish, or didn't get the codes working, you must submit your code to the Moodle Assignment for similarity checking.*
4. For your preliminary and lab works to be graded you must attend the lab.

4. Cleanup

1. After saving any files that you might want to have in the future to your own storage device, erase all the files you created from the computer in the lab.
2. When applicable put back all the hardware, boards, wires, tools, etc where they came from.
3. Clean your lab desk and it ready for the next group.

Lab Policies

1. You can do the lab only in your section. Missing your section time and attending another day is not allowed.
2. The questions asked by the TA will have an effect on your lab score.
3. Lab score will be reduced to 0 if the code is not submitted for similarity testing, or if it is plagiarized. MOSS-testing will be done, to determine similarity rates. Trivial changes to code will not hide plagiarism from MOSS—the algorithm is quite sophisticated and powerful. Please also note that obviously you should not use any program available on the web, or in a book, etc. since MOSS will find it. The use of the ideas we discussed in the classroom is not a problem.
4. You must be in lab, working on the lab, from the time lab starts until your work is finished and you leave.
5. No cell phone use during lab.
6. Internet usage is permitted only to lab-related technical sites.